Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026

EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE STAGE PROFESSIONNEL

Spécialité: .....

Intitulé

.....

Lieu du stage **Réalisé par** **Encadré par**

Dédicace

Remerciements

Résumé

Ce rapport présente la conception et la réalisation d'une application web interne dédiée à la gestion des demandes techniques. Le projet couvre l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes, avec une historisation des changements importants. L'architecture associe un frontend Angular, une API REST Spring Boot et une base PostgreSQL, selon une organisation modulaire séparant présentation, application, domaine et persistance. La sécurité repose sur l'authentification JWT, le contrôle d'accès fondé sur les rôles et des contrôles métier contextuels. Une assistance IA intégrée via Groq propose une catégorie et une priorité à partir du titre et de la description ; le Responsable technique conserve la décision finale. La validation s'appuie sur des tests automatisés backend et frontend, une intégration locale du parcours IA et une vérification fonctionnelle humaine. Le rapport distingue les résultats mesurés sur le périmètre V1 des limites, notamment le caractère exploratoire du benchmark IA et l'absence de portail client.

Mots-clés : génie logiciel ; API REST ; Angular ; Spring Boot ; JWT ; intelligence artificielle.

Abstract

This report presents the design and implementation of an internal web application for managing technical requests. The project covers request registration, qualification, assignment, processing, resolution and closure, with the history of significant changes. The architecture combines an Angular frontend, a Spring Boot REST API and a PostgreSQL database, using modular responsibilities for presentation, application, domain and persistence. Security relies on JWT authentication, role-based access control and contextual business checks. An AI assistance feature integrated through Groq proposes a category and a priority from the request title and description ; the Technical Manager remains responsible for the final decision. Validation combines automated backend and frontend tests, a local integration of the AI workflow and human functional verification. The report distinguishes measured results within the V1 scope from remaining limitations, including the exploratory nature of the AI benchmark and the absence of a customer portal.

Mots-clés

Sommaire

Introduction générale **1**

1 Contexte et cadrage du projet **2**
1.1 Cadre professionnel et informations disponibles 3
1.2 Contexte fonctionnel 3
1.3 Problématique et objectifs 3
1.4 Étude de l'existant et limites du périmètre 3
1.5 Acteurs et responsabilités 4
1.6 Démarche et organisation du travail 4
1.7 Synthèse du cadrage 4

2 Analyse et conception **5**
2.1 Spécification des besoins 6
2.2 Conception fonctionnelle 7
2.3 Conception technique 8
2.4 Conception de la base de données 9

3 Réalisation et validation **12**
3.1 Vue d'ensemble de la solution réalisée 13
3.2 Parcours complet d'une demande 13
3.3 Administration et contrôle des accès 16
3.4 Assistance IA et validation humaine 16
3.5 Tests et validation 17
3.6 Limites et perspectives 17

Conclusion générale **19**

Annexes **20**

Références **21**

Table des figures

1 Connexion à l'application 13
2 Création et qualification initiale d'une demande 14
3 Liste filtrable des demandes 14
4 Traitement d'une demande affectée 15
5 Clôture après examen de la résolution 15
6 Gestion des utilisateurs internes 16
7 Assistance IA lors de la qualification 17

Liste des tableaux

1 Capacités fonctionnelles de la V1 7
2 Dictionnaire de données synthétique 11
3 Niveaux de validation et portée 18

Introduction générale

Le stage porte sur la conception et la réalisation d'une application web interne destinée à organiser les demandes techniques reçues des clients. L'enjeu est de qualifier les demandes, attribuer les responsabilités, sécuriser les accès, suivre les transitions et conserver une trace des changements.

La problématique est la suivante : comment construire une solution simple à exploiter, maintenable et contrôlée pour accompagner tout le cycle de vie d'une demande, tout en conservant une décision humaine lorsque l'assistance IA est sollicitée ?

Le projet vise une architecture Angular, Spring Boot et PostgreSQL, structurée en modules et protégée par JWT et RBAC. Le rapport présente d'abord le contexte et le cadrage, puis l'analyse et la conception, avant de décrire la réalisation et la validation. Les informations administratives absentes restent signalées comme éléments à compléter.

Chapitre 1

Contexte et cadrage du projet

1.1 Cadre professionnel et informations disponibles

Le projet étudié concerne une application interne de gestion des demandes techniques. Les sources disponibles décrivent précisément le besoin logiciel et la solution réalisée, mais elles ne fournissent pas les informations administratives nécessaires pour présenter l'organisme d'accueil, son secteur d'activité, l'équipe ou l'encadrement. Ces éléments sont donc laissés à compléter : nom officiel de l'organisme, activité, service d'accueil, encadrants et dates du stage.

Cette absence ne doit pas être compensée par une description générique. Le rapport distingue le contexte logiciel, vérifiable dans le projet, du contexte professionnel, qui devra être renseigné à partir des informations communiquées par l'étudiant ou l'organisme d'accueil.

1.2 Contexte fonctionnel

Une demande technique naît lorsqu'un client rencontre un problème ou formule un besoin nécessitant l'intervention d'une équipe technique. La difficulté n'est pas limitée à la saisie initiale. Une demande doit être suffisamment décrite pour être comprise, qualifiée pour être priorisée, affectée à une personne responsable, puis suivie jusqu'à une résolution acceptée ou à une clôture justifiée.

Le projet vise donc à fournir un espace commun où les rôles, les décisions et l'état courant d'une demande restent visibles. La V1 se concentre sur les utilisateurs internes : le client est représenté comme une donnée métier, mais ne se connecte pas à l'application.

1.3 Problématique et objectifs

La problématique retenue est la suivante : comment construire une application interne capable de piloter le cycle de vie d'une demande technique, en conciliant simplicité d'utilisation, contrôle des droits, cohérence des données et traçabilité des décisions ?

Les objectifs sont d'enregistrer une demande, donner à chaque rôle une vision cohérente, formaliser les transitions, conserver l'historique, protéger l'API, proposer une assistance IA facultative et rendre le fonctionnement vérifiable par des tests et une validation humaine. L'objectif n'est pas de multiplier les fonctionnalités, mais de construire un parcours cohérent sur un périmètre V1 maîtrisé.

1.4 Étude de l'existant et limites du périmètre

Les sources du projet ne documentent pas un processus métier antérieur observé dans une entreprise. Il serait donc incorrect d'affirmer que l'application remplace un outil précis ou corrige

des délais mesurés. L'étude de l'existant est limitée aux besoins identifiés, aux rôles, aux étapes du cycle et aux exclusions de la V1.

Le périmètre comprend les demandes internes, les clients associés, la qualification, l'affectation, le traitement, la résolution, la clôture, le refus de résolution, l'annulation avec motif, l'historisation et l'administration des utilisateurs. Il exclut le portail client, le compte client, un module autonome de gestion de clients, les sous-catégories, le statut d'attente, les commentaires, la suppression fonctionnelle et la décision IA automatique.

Cette délimitation maîtrise le risque et évite de confondre une perspective future avec une fonction livrée.

1.5 Acteurs et responsabilités

Le Responsable technique enregistre et qualifie les demandes, organise l'affectation et examine les résolutions. L'Agent technique traite les demandes qui lui sont affectées, renseigne la solution et déclare la résolution. L'Administrateur gère les comptes internes et leurs rôles métier ; ce rôle administratif ne donne pas automatiquement accès aux opérations métier. Un utilisateur peut cumuler plusieurs rôles, ce qui explique la vérification de l'ensemble des rôles et du contexte de la demande.

1.6 Démarche et organisation du travail

La démarche suit une progression de génie logiciel : formalisation du besoin et du périmètre, modélisation des règles de cycle de vie, conception de l'architecture et des données, réalisation, puis validation. La validation distingue les tests automatisés, les vérifications d'intégration locale et l'observation humaine des parcours dans le navigateur.

1.7 Synthèse du cadrage

Le cadrage fait apparaître une tension centrale : l'application doit guider le processus sans empêcher les décisions métier. Les statuts, les rôles et l'historique structurent le fonctionnement ; l'assistance IA réduit l'effort de qualification mais reste une proposition. Le chapitre suivant montre comment cette tension a été traduite en exigences, modèles et choix d'architecture.

Chapitre 2

Analyse et conception

Ce chapitre présente le passage du besoin métier à une conception structurée. L'analyse commence par les acteurs et les attentes du système, puis décrit les scénarios et les règles du cycle de vie. Elle aboutit à l'architecture, à la sécurité, à l'intégration de l'assistance IA et au modèle de données. Les éléments présentés ici décrivent des décisions de conception ; la réalisation et les résultats de validation sont traités au chapitre 3.

2.1 Spécification des besoins

Acteurs et responsabilités. L'application est réservée aux utilisateurs internes. Le Responsable technique enregistre et qualifie les demandes, valide ou modifie leur catégorie et leur priorité, affecte ou réaffecte les demandes, suit leur avancement, examine une résolution et peut clôturer, refuser ou annuler une demande selon son état. L'Agent technique consulte les demandes qui lui sont affectées, commence le traitement, renseigne la description du traitement et la solution, puis déclare la demande résolue. L'Administrateur crée et modifie les comptes, active ou désactive les utilisateurs et gère les rôles métier autorisés. Une même personne peut cumuler plusieurs rôles.

Le Client n'est pas un utilisateur de l'application. Il constitue une entité métier associée à une demande. Cette distinction est importante : elle permet de conserver les informations nécessaires au suivi sans introduire dans la V1 un portail, une authentification ou des droits d'accès pour les clients.

Besoins fonctionnels. Les besoins sont regroupés par capacités métier plutôt que par identifiants de règles. Le premier groupe concerne l'accès : un utilisateur interne doit pouvoir se connecter et accéder uniquement aux fonctions correspondant à ses rôles. Le deuxième concerne les demandes : le Responsable technique doit pouvoir enregistrer une demande avec un client, une catégorie, une priorité et une description, puis consulter et filtrer les demandes. Le troisième concerne le workflow : une demande peut être affectée, traitée, déclarée résolue, clôturée, refusée ou annulée dans les conditions prévues. Le quatrième concerne l'administration des utilisateurs. Le dernier concerne l'assistance IA, qui propose seulement une catégorie et une priorité après une action explicite.

Ces besoins décrivent ce que le système doit permettre, et non la manière de le réaliser. Par exemple, « réserver l'accès aux utilisateurs autorisés » est un besoin de protection ; le JWT et le contrôle d'accès basé sur les rôles sont des moyens techniques étudiés plus loin. De même, « conserver les changements importants » est un besoin de traçabilité ; l'entité d'historique et la transaction sont des choix de conception.

Besoins non fonctionnels. La sécurité est nécessaire parce que les fonctions sont réservées à des utilisateurs internes et que les actions dépendent du rôle et parfois du contexte de la demande. La cohérence des données est nécessaire parce qu'une transition et son historique doivent rester synchronisés. La traçabilité est nécessaire pour comprendre les changements importants sans réécrire un événement déjà enregistré.

TABLE 1 – Capacités fonctionnelles de la V1

Capacité	Comportement attendu
Accès interne	Authentifier un utilisateur et orienter son accès selon ses rôles.
Gestion des demandes	Enregistrer, consulter, filtrer et suivre les demandes techniques.
Qualification	Définir une catégorie et une priorité validées par le Responsable technique.
Workflow	Affecter, traiter, résoudre, clôturer, refuser ou annuler selon l'état courant.
Administration	Gérer les comptes, l'état actif et les rôles métier autorisés.
Assistance IA	Proposer une catégorie et une priorité, sans décision automatique.

La maintenabilité est recherchée par la séparation des responsabilités et des contrats d'échange. L'ergonomie et le responsive design répondent à la diversité des écrans et à la nécessité de comprendre rapidement l'état d'une demande. Enfin, la continuité manuelle est une exigence fonctionnelle autour de l'IA : une panne ou une indisponibilité du fournisseur ne doit pas empêcher la qualification classique. Aucun seuil de performance ou de disponibilité non mesuré n'est introduit dans le rapport.

2.2 Conception fonctionnelle

Cycle de vie. Une demande commence dans l'état `NOUVELLE`. Une affectation la place dans l'état `ASSIGNEE`, puis l'Agent peut commencer le traitement et la faire passer à `EN_COURS`. Lorsque la solution est renseignée, l'Agent déclare la demande `RESOLUE`. Le Responsable technique peut l'accepter et la clôturer, ou refuser la résolution et la remettre en traitement. Une demande peut être annulée depuis `NOUVELLE`, `ASSIGNEE` ou `EN_COURS`, avec un motif obligatoire. `CLOTUREE` et `ANNULEE` sont terminaux.

La réaffectation illustre une règle qui ne serait pas visible avec une simple liste de statuts. Depuis `ASSIGNEE`, une nouvelle affectation conserve cet état. Depuis `EN_COURS`, la réaffectation à un autre Agent revient à `ASSIGNEE`, afin que le nouvel Agent commence explicitement le traitement. La solution et la description du traitement ne sont pas effacées par cette opération.

Scénario d'enregistrement et de qualification. Le Responsable technique renseigne le titre et la description, recherche un client et sélectionne un client existant ou saisit les informations minimales d'un nouveau client. Il définit ou valide ensuite la catégorie et la priorité, puis enregistre la demande. Le système crée une référence, associe le créateur et le client, initialise

le statut à `NOUVELLE` et conserve l'événement de création. Une erreur de validation ou l'absence d'un client exploitable empêche l'enregistrement ; elle ne produit pas une demande partielle.

Scénario de traitement et de résolution. Le Responsable technique affecte la demande à un Agent. Celui-ci démarre le traitement, renseigne la description et la solution, puis déclare la demande résolue. Le Responsable examine alors la résolution. L'alternative principale est le refus : la demande revient à `EN_COURS`, la date de résolution est réinitialisée et l'historique conserve la trace de la décision. Une tentative d'action depuis un état incompatible est rejetée.

Scénario d'assistance IA. Le Responsable technique clique explicitement sur « Analyser avec l'IA ». Le système transmet le titre et la description au service d'analyse. Une réponse valide propose une catégorie et une priorité qui restent modifiables dans le formulaire. Si l'analyse échoue ou est indisponible, le Responsable poursuit la qualification manuellement. La proposition ne devient jamais une décision métier sans validation humaine.

Les diagrammes fonctionnels validés ont chacun un rôle distinct. Le cas d'utilisation répond à la question « qui utilise quelle capacité ? ». Le diagramme d'activité montre les transitions du cycle. La séquence d'enregistrement détaille le scénario nominal et l'alternative du client existant ou nouveau. Les autres séquences, plus spécialisées, peuvent compléter l'analyse en annexe si leur taille nuit à la lecture du corps du rapport.

2.3 Conception technique

De la contrainte au choix architectural. Le projet doit fournir une interface web, centraliser les règles métier, persister des données relationnelles et intégrer un fournisseur IA externe. Une architecture Angular, API REST Spring Boot et PostgreSQL répond à ces besoins en séparant l'interface, le traitement serveur et le stockage. Le backend est organisé comme un monolithe modulaire : le déploiement reste simple pour le périmètre V1, tandis que les responsabilités sont isolées autour de la gestion des demandes, de l'identité et de l'administration, de la sécurité et de l'IA. Aucune comparaison détaillée avec les microservices n'est nécessaire pour justifier ce choix.

Dans Angular, les fonctionnalités sont regroupées par domaine et les éléments transversaux sont placés dans les espaces communs. Les formulaires réactifs représentent les données saisies et les services d'accès à l'API évitent de placer les règles métier dans les composants. Dans Spring Boot, les Controllers reçoivent les contrats HTTP, les Services d'application orchestrent les cas d'utilisation, le domaine porte les invariants et les Repositories isolent la persistance. Les DTO séparent les représentations d'entrée et de sortie des entités JPA.

SOLID et techniques effectivement observables. La séparation Controller, Service, domaine, Repository et DTO illustre le principe de responsabilité unique (SRP) : chaque responsabilité principale possède un emplacement identifiable. Cette observation ne signifie pas que toutes les classes respectent parfaitement SRP, ni que les cinq principes SOLID sont appliqués uniformément.

Le service d'analyse dépend de 'AnalyseIaPort', tandis que 'GroqAnalyseIaAdapter' fournit l'intégration concrète. Cette organisation illustre le principe d'inversion des dépendances (DIP) et la technique Port/Adapter : le cœur métier ne dépend pas directement du SDK ou du protocole du fournisseur. Le Repository Pattern est observable dans les interfaces de persistance utilisées par les services. Les DTO constituent une séparation de contrat entre HTTP et entités persistées ; ils ne sont pas présentés comme un pattern universel, mais comme une technique de découplage utile dans ce projet.

Le State Pattern n'a pas été retenu pour le workflow. Pour la V1, un enum de statut et des opérations métier explicites suffisent à vérifier les transitions. Cette décision réduit la complexité tant que le nombre d'états et de règles reste limité ; elle pourrait être réévaluée si le cycle de vie s'étend fortement.

Sécurité conçue. Le besoin de protection conduit d'abord à distinguer l'authentification et l'autorisation. L'authentification vérifie l'identité d'un utilisateur actif. Le JSON Web Token (JWT) est ensuite utilisé comme jeton d'accès transmis dans l'en-tête HTTP. Le contrôle d'accès basé sur les rôles (RBAC) organise les permissions générales selon les rôles Responsable technique, Agent technique et Administrateur. Il est complété par des contrôles métier contextuels : un Agent ne traite que ses demandes affectées, les transitions dépendent de l'état et l'auto-désactivation est interdite.

Les mots de passe sont transformés par 'PasswordEncoder' et les secrets sont fournis par une configuration externe. La session Angular peut être restaurée depuis le stockage de session, mais cette facilité d'interface ne remplace pas la vérification backend de la signature du JWT, du compte actif, des rôles courants et de l'autorisation de l'action. La conception conserve ainsi l'API comme autorité de sécurité.

Assistance IA. La conception suit le flux : titre et description, analyse, proposition d'une catégorie et d'une priorité, puis validation ou modification humaine. Le port d'intégration isole le fournisseur Groq du cœur métier. Cette séparation permet de contrôler le format de la réponse et de maintenir la qualification manuelle lorsque le système externe est indisponible. Les résultats du benchmark et la validation locale appartiennent au chapitre 3 ; ils ne sont pas utilisés ici pour transformer une conception en preuve de qualité générale.

Les diagrammes de composants et de conception backend sont destinés à représenter ces responsabilités et leurs dépendances. Ils doivent être lus comme deux niveaux complémentaires : le premier montre les frontières applicatives et les systèmes externes ; le second montre les rôles des classes techniques à l'intérieur du backend.

2.4 Conception de la base de données

La base relationnelle matérialise les objets métier et les contraintes qui protègent leur cohérence. Le modèle ne cherche pas à stocker toutes les décisions dans une seule table : les utilisateurs, les clients, les demandes et les événements d'historique ont des responsabilités

distinctes. Les relations sont vérifiées à la fois dans les entités JPA et dans la migration Flyway V1 initial schema.

‘Utilisateur‘ possède un identifiant généré par séquence, un nom, un email, un état actif, un hash de mot de passe et un ensemble de rôles. L’email est non nul et unique dans la base ; sa normalisation est assurée par le domaine. Les rôles sont stockés dans une table d’association dont la clé primaire combine l’identifiant utilisateur et le code du rôle.

‘Client‘ possède un identifiant généré par séquence, un nom, un email et un téléphone, tous obligatoires. L’email du client n’est pas déclaré unique, car aucune règle métier validée ne l’impose. ‘DemandeTechnique‘ référence obligatoirement un client et un créateur, et peut posséder au plus un Agent affecté. Sa référence est non nulle, unique et non modifiable après création. Les catégories, priorités et statuts sont persistés comme codes textuels correspondant aux enums.

HistoriqueDemande conserve la date, le type d’événement, les anciennes et nouvelles valeurs éventuelles, la demande concernée et l’auteur. La relation vers la demande et l’auteur est obligatoire. L’historique est fonctionnellement immuable : une correction produit un nouvel événement plutôt que de réécrire l’ancien. Les dates de création, modification, résolution, clôture et annulation sont des instants ; la migration les stocke en TIMESTAMP WITH TIME ZONE, cohérent avec Instant côté Java (PostgreSQL Global Development Group 2026).

Le diagramme des entités persistantes complète ce dictionnaire en montrant les cardinalités : une demande est liée à un client et à un créateur, un agent peut être absent, et une demande possède plusieurs événements d’historique. Le diagramme de conception backend ajoute les repositories, services, DTO et contrôleurs ; il ne remplace pas le schéma relationnel, mais explique comment l’application y accède.

En résumé, la conception relie chaque besoin à une responsabilité identifiable : les cas d’utilisation structurent le comportement, l’architecture répartit les responsabilités, la sécurité protège les opérations et le modèle de données garantit les relations essentielles. Les rendus UML détaillés sont conservés dans les sources validées ; leur intégration graphique dépend de la disponibilité d’un renderer PlantUML local.

TABLE 2 – Dictionnaire de données synthétique

Entité	Attribut	Type	Contrainte et signification
Utilisateur	id	Long / BIGINT	Clé primaire, séquence.
Utilisateur	email	String / VARCHAR	Obligatoire, normalisé et unique.
Utilisateur	actif	boolean / BOOLEAN	Obligatoire ; compte autorisé ou désactivé.
Utilisateur	roles	Set<Role>	Association ; plusieurs rôles possibles.
Client	id	Long / BIGINT	Clé primaire, séquence.
Client	nom, email, telephone	String / VARCHAR	Trois informations obligatoires ; email non unique.
DemandeTechnique	reference	String / VARCHAR	Obligatoire, unique et immuable ; référence métier.
DemandeTechnique	titre, description	String / TEXT	Obligatoires ; contenu de la demande.
DemandeTechnique	categorie	Categorie / VARCHAR	Obligatoire ; valeur de la liste V1.
DemandeTechnique	priorite	Priorite / VARCHAR	Obligatoire ; BASSE à CRITIQUE.
DemandeTechnique	statut	Statut / VARCHAR	Obligatoire ; état du cycle de vie.
DemandeTechnique	agentAffecte	Utilisateur / FK	Optionnel ; au plus un Agent affecté.
DemandeTechnique	dates métier	Instant / timestamptz	Création et modification obligatoires ; résolution, clôture et annulation optionnelles.
HistoriqueDemande	typeEvenement	String / VARCHAR	Obligatoire ; nature du changement.
HistoriqueDemande	ancienne/nouvelle valeur	String / TEXT	Optionnelles ; valeurs avant et après.
HistoriqueDemande	demande, auteur	FK	Obligatoires ; origine et responsable de l'événement.

Chapitre 3

Réalisation et validation

3.1 Vue d'ensemble de la solution réalisée

Angular fournit l'application web et les parcours utilisateur. Spring Boot expose l'API et applique les règles métier. PostgreSQL conserve les données et l'historique. Cette répartition permet de faire évoluer l'interface sans déplacer les invariants métier dans le navigateur.

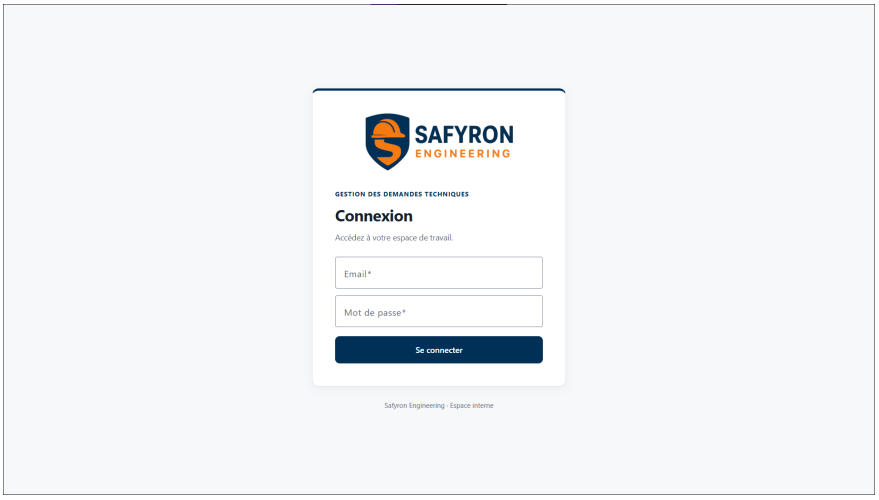


FIGURE 1 – Connexion à l'application

Après authentification, le frontend conserve le token et le snapshot minimal utile à l'orientation de l'interface. Les autorisations effectives restent décidées par l'API.

3.2 Parcours complet d'une demande

Le parcours commence par la création et la qualification. Le Responsable technique renseigne un titre, une description, un client, une catégorie et une priorité. Il peut demander une suggestion IA, mais garde la possibilité de modifier les valeurs avant l'enregistrement.

La liste permet ensuite de retrouver les demandes par filtres, tri et pagination côté serveur. Après affectation, l'Agent traite la demande, renseigne la solution et déclare la résolution. Le Responsable examine cette proposition : il clôture la demande ou la refuse pour demander une reprise.

Ces écrans illustrent une propriété importante : l'interface présente des actions dépendantes du rôle et du statut, mais cette présentation ne remplace pas le contrôle côté serveur.

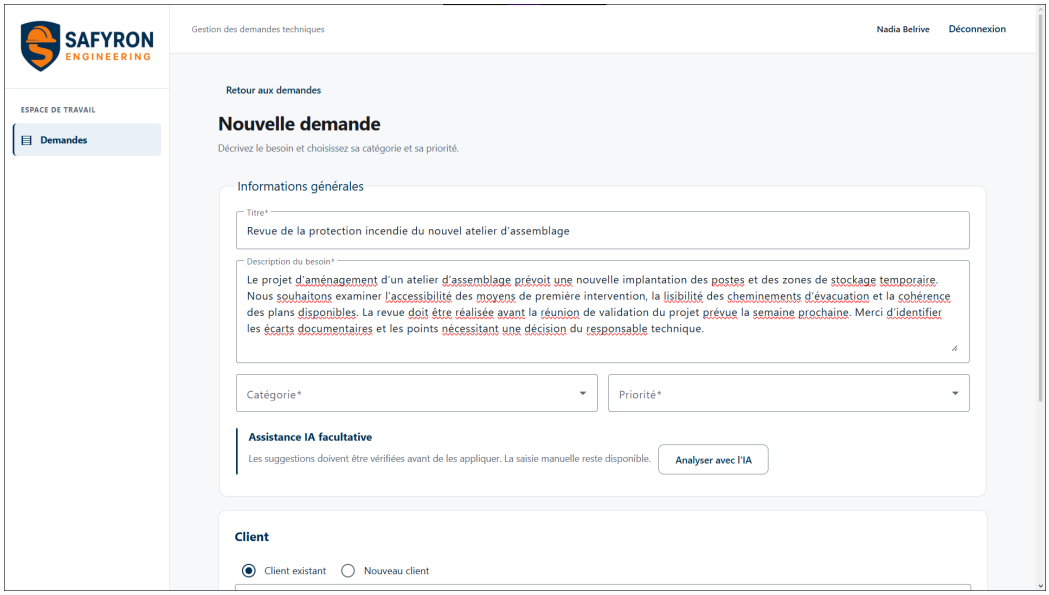


FIGURE 2 – Création et qualification initiale d'une demande

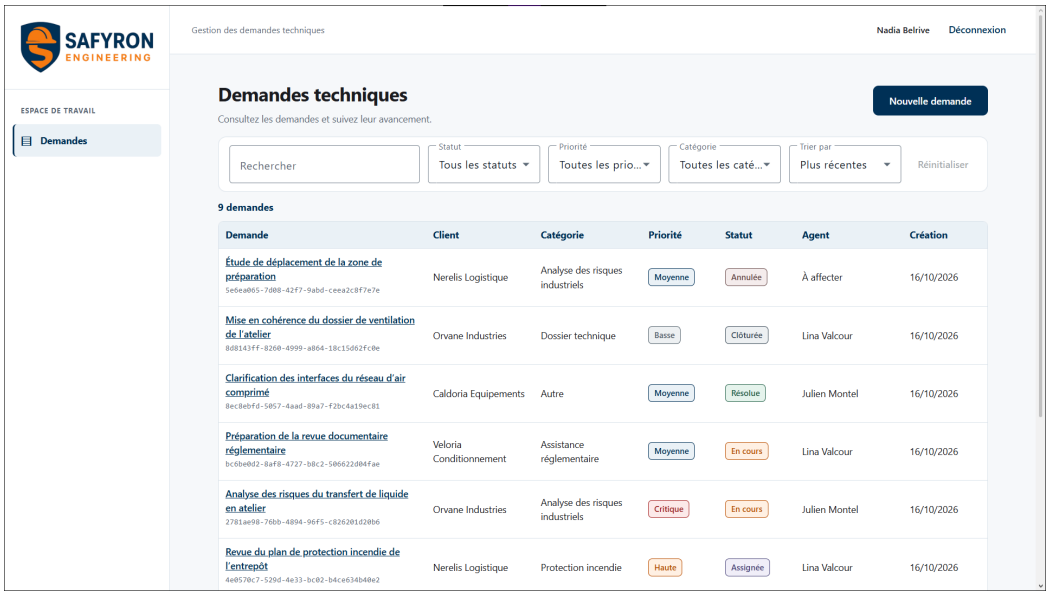


FIGURE 3 – Liste filtrable des demandes

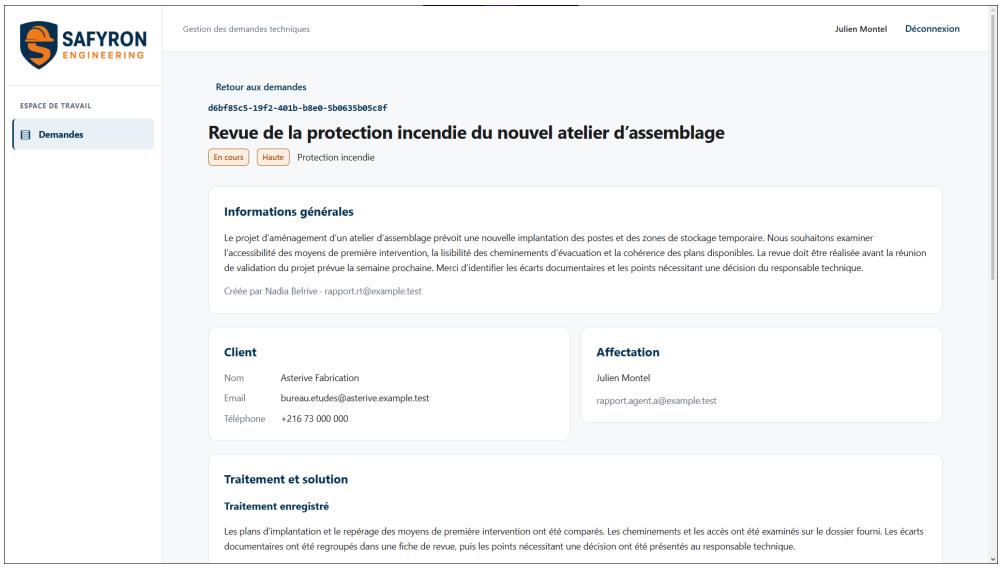


FIGURE 4 – Traitement d'une demande affectée

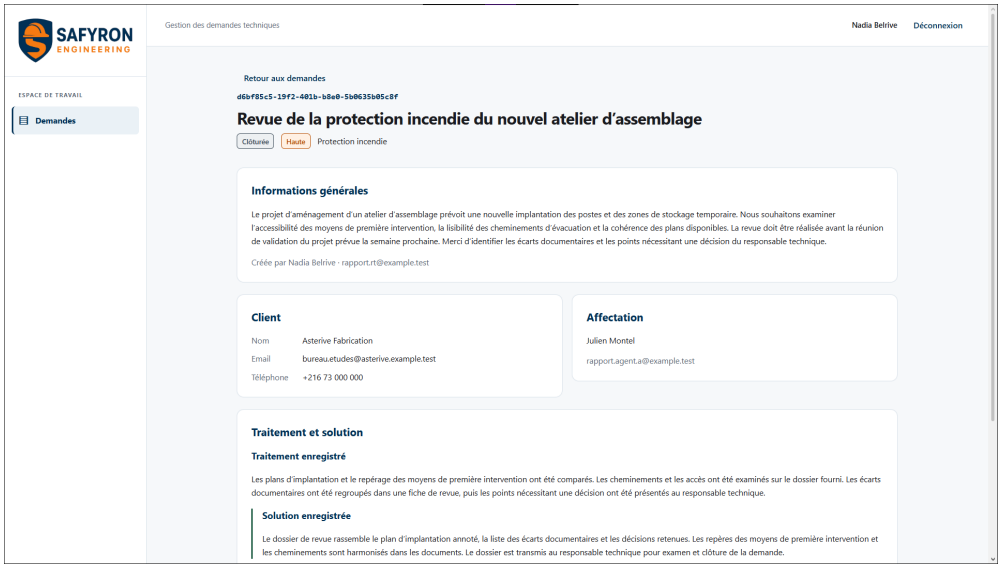


FIGURE 5 – Clôture après examen de la résolution

3.3 Administration et contrôle des accès

L'administration permet de créer et modifier les comptes, de gérer leur état actif et de remplacer leurs rôles métier. L'auto-désactivation du compte courant est refusée par l'API avec une réponse 409 et l'interface conserve un retour compréhensible.

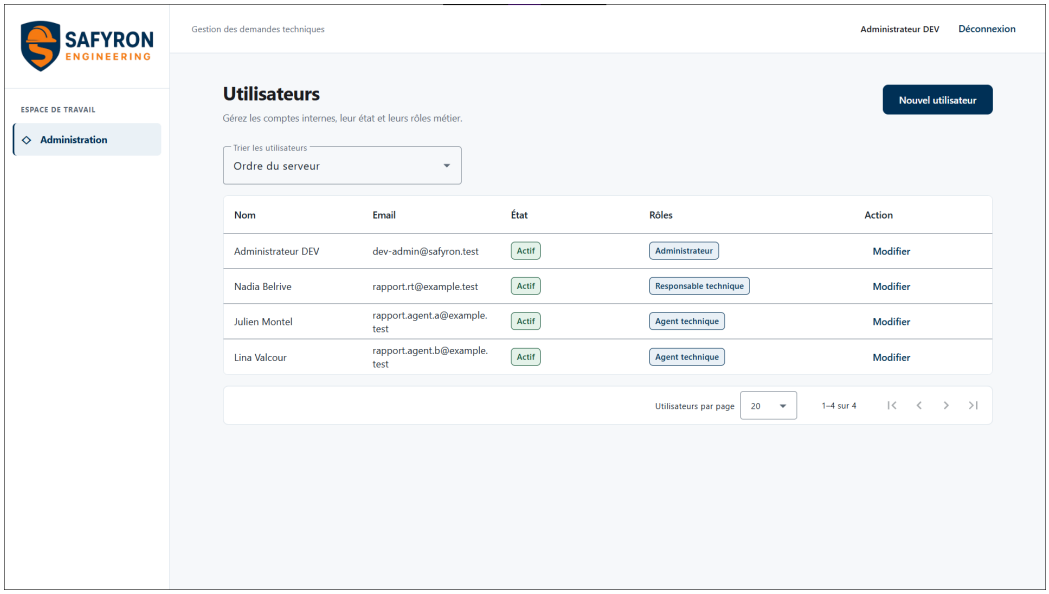


FIGURE 6 – Gestion des utilisateurs internes

Le backend relit l'utilisateur et ses rôles à chaque requête protégée. Un rôle supprimé ou un compte désactivé ne conserve donc pas ses droits simplement parce qu'un snapshot ancien existe dans le navigateur.

3.4 Assistance IA et validation humaine

L'analyse IA est déclenchée depuis le formulaire de nouvelle demande. Le frontend envoie uniquement le titre et la description à l'endpoint backend. Le serveur appelle l'adaptateur Groq via ‘AnalyseIaPort‘, contrôle la réponse puis renvoie une proposition temporaire. La catégorie et la priorité ne sont jamais appliquées automatiquement.

Une modification du titre ou de la description invalide la suggestion. Lorsque la clé Groq est absente, l'API renvoie une indisponibilité contrôlée et le formulaire reste utilisable manuellement. Le benchmark exploratoire porte sur 24 exemples : sorties valides dans les 24 cas, catégorie correcte dans 23 cas et priorité dans 22 cas. Ces valeurs ne permettent pas d'annoncer une performance générale ni une supériorité universelle d'un fournisseur.

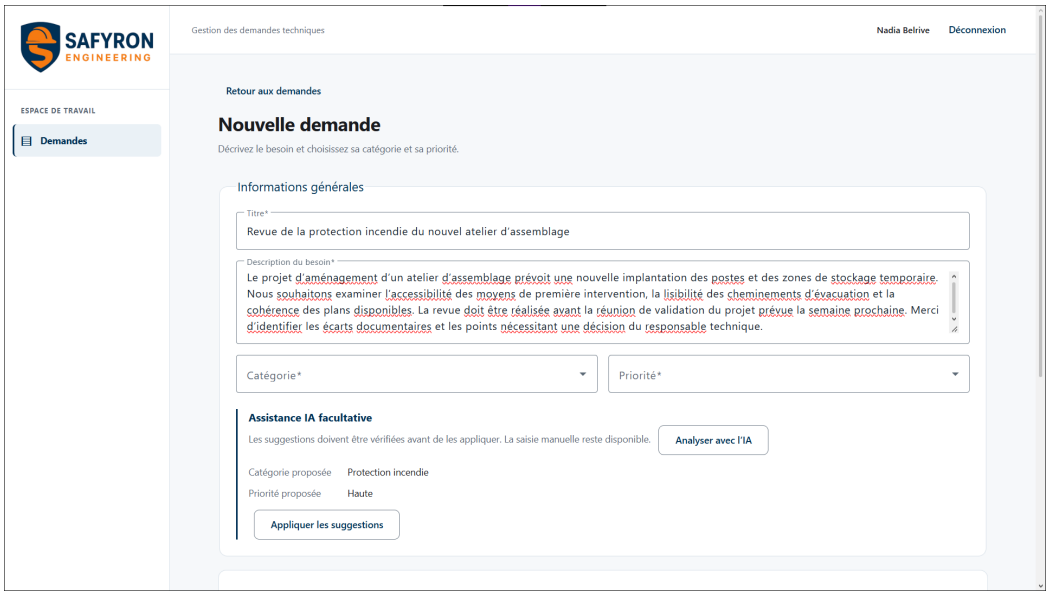


FIGURE 7 – Assistance IA lors de la qualification

3.5 Tests et validation

La validation backend atteint 270 tests réussis avec ‘clean test‘ et ‘clean verify‘, sans échec, erreur ou test ignoré. Les tests IA hors ligne utilisent H2 et un stub HTTP local ; ils ne constituent pas un appel Groq. Le frontend a été vérifié par ‘npm ci‘, ‘npm run build‘ et ‘npm test ––watch=false‘, avec 184 tests réussis dans 15 fichiers pour la clôture IA.

La validation fonctionnelle humaine a couvert la connexion, la liste, la création, la sélection client, les transitions de demande, l’administration, l’auto-désactivation et l’assistance IA réelle en local. Une revue responsive et accessibilité a également été effectuée sur plusieurs tailles. Ces observations ne remplacent pas une campagne de test de charge ou une validation de production.

3.6 Limites et perspectives

Le rapport ne prétend pas mesurer la performance en production, la résistance à la charge ou la qualité IA sur un corpus représentatif du long terme. Le périmètre reste interne et le client n’interagit pas directement avec la plateforme. La session navigateur utilise ‘sessionStorage‘, ce qui permet une restauration après rechargement mais conserve les risques propres à un stockage accessible au JavaScript du navigateur.

Les perspectives cohérentes avec la V1 sont l’élargissement du corpus d’évaluation IA, l’ajout de mesures répétées et l’amélioration de l’observabilité. Toute extension du workflow, du

TABLE 3 – Niveaux de validation et portée

Validation	**Ce qu'elle permet d'affirmer**
Tests backend	Les comportements couverts par 270 tests réussissent dans l'environnement vérifié.
Tests frontend	La construction et les scénarios automatisés couverts réussissent.
Intégration locale IA	Le parcours Angular vers Spring Boot et Groq a fonctionné dans les conditions observées.
Validation humaine	Les parcours et l'ergonomie vérifiés ne présentent pas de blocage constaté dans le périmètre revu.
Benchmark IA	Résultats exploratoires sur 24 exemples, non généralisables.

portail client ou du déploiement devra être précédée d'une analyse de besoin et d'une validation dédiée.

Conclusion générale

Le stage a conduit à une application V1 couvrant l'enregistrement, la qualification, l'affectation, le traitement et le suivi des demandes techniques. La séparation entre interface Angular, API Spring Boot et persistance PostgreSQL fournit une base lisible et évolutive. La sécurité combine authentification JWT, rôles métier et contrôles contextuels.

L'assistance IA facultative via Groq propose une catégorie et une priorité, mais la décision finale reste modifiable et validée par le Responsable technique. Les tests automatisés et les validations humaines montrent que le périmètre V1 est fonctionnel dans les conditions vérifiées.

Les limites concernent le périmètre interne, l'absence de portail client et le caractère exploratoire du benchmark IA. Des travaux ultérieurs pourraient élargir le corpus d'évaluation, renforcer l'observabilité et étudier les besoins de déploiement, sous réserve de validations supplémentaires.

Annexes

Références

PostgreSQL Global Development Group (2026). *Date/Time Types*. URL : https://www.postgresql.org/docs/current/datatype-datetime.html (visité le 11/10/2026).

Table des matières détaillée

Introduction générale **1**

1 Contexte et cadrage du projet **2**
1.1 Cadre professionnel et informations disponibles 3
1.2 Contexte fonctionnel 3
1.3 Problématique et objectifs 3
1.4 Étude de l'existant et limites du périmètre 3
1.5 Acteurs et responsabilités 4
1.6 Démarche et organisation du travail 4
1.7 Synthèse du cadrage 4

2 Analyse et conception **5**
2.1 Spécification des besoins 6
2.2 Conception fonctionnelle 7
2.3 Conception technique 8
2.4 Conception de la base de données 9

3 Réalisation et validation **12**
3.1 Vue d'ensemble de la solution réalisée 13
3.2 Parcours complet d'une demande 13
3.3 Administration et contrôle des accès 16
3.4 Assistance IA et validation humaine 16
3.5 Tests et validation 17
3.6 Limites et perspectives 17

Conclusion générale **19**

Annexes **20**

Références **21**